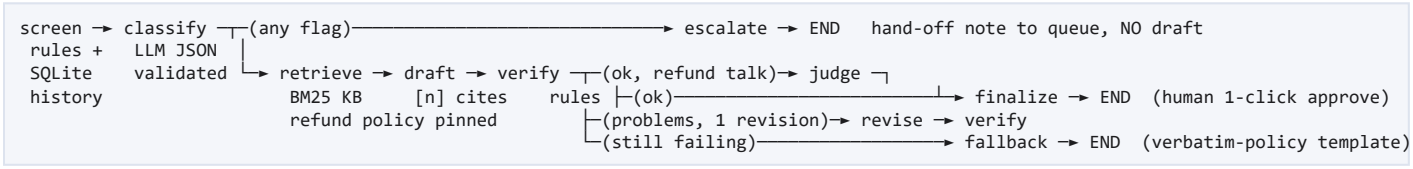
Agentic Architect Challenge: System Architecture

Lawrance Tan · github.com/weilun9/agentic-architect-challenge · Python 3.12, LangGraph, Ollama *qwen3.5:9b* locally (OpenAI/Gemini by env var) · 85 offline tests, CI on Linux + Windows

Shared layer

llm.py: one chat-model factory, `call()` (retry + backoff, latency/tokens traced) and `call_json()` (Pydantic validation, one repair prompt, then `LLMError`). **obs.py**: every step appends one JSON line (run_id, email id, step, status, ms, tokens, guardrail) → `aac trace-summary` gives p50/p95, errors, tokens and guardrail counts per step. Callers **fail closed** on `LLMError`.

Part 1: Support-email triage (LangGraph state machine)



- **Flag before drafting**: deterministic keyword rules (data loss / outage / breach) and contact count > 3 in the 7 days up to the email run first, with no LLM. The classifier adds multi-label categories plus *risk_signals* for paraphrases. Either source escalates, and the gate sits before retrieval and drafting.
- **Queues**: security-incident › tier2-technical › customer-care-lead › human-triage.
- **Fail closed**: classifier down or invalid JSON → human. An exception in one email is contained, and the batch continues (thread pool).
- **Refund hallucination, three layers**: (1) policy pinned and cited per claim; (2) rules: every number + unit must be in the cited passage, refund terms must cite the refund policy, no "we will refund / you are eligible" unless conditional or negated; (3) LLM judge reads each refund sentence against the full policy (no verdict = reject). Still failing → template that quotes the policy verbatim.
- **Live result** (10 emails): all 4 escalations caught (1 by the LLM signal only); a refund draft that said "you are eligible" was blocked twice → template.

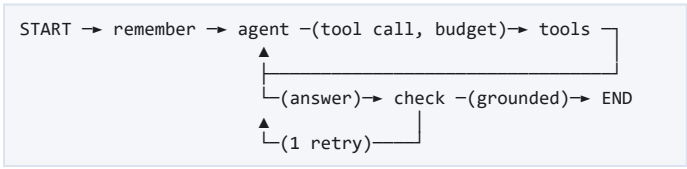
Part 2: Scrape and summarise: bottleneck and fix

Bottleneck: the original sends `soup.get_text()` of the whole page in **one prompt**. Evidence: on PEP 8 Ollama logged `truncating input prompt limit=2050 prompt=10903` (the model saw 19% and summarised from memory). On Wikipedia, with no User-Agent or status check, it summarised the **403 bot-policy page**.

Fetch	Extract	Summarise (map-reduce)	Conciseness guardrail
timeouts, retry/backoff on 429/5xx, UA, raise_for_status, content-type check, streamed 5 MB cap; headless Chromium for JS shells (optional)	drop script/style/nav/footer/cookie/share (whole-token class match); <article>/<main> or densest low-link-density block; dedupe lines	~1.5k-token chunks condensed in a bounded thread pool → merged (tree-reduce if needed); >24 chunks sampled evenly and reported; failed chunks skipped	budget in prompt + output-token cap → `check_concise` (words, bullets, preamble) → one targeted rewrite → deterministic sentence-boundary truncation. Always within budget.

PEP 8: 6 chunks, 7 calls, whole page, 110 words, 36 s (same time as the naive call). Wikipedia (28k tokens): 20 chunks, first summary 178 words → guardrail rewrite → 94 words.

Part 3: Document agent with memory and a tool



- Single ~600-token doc in the system prompt (no retrieval step that can miss).
- **calculator**: AST whitelist, no eval/names, exponent and size caps. The model decides; the prompt says arithmetic only.
- **Memory**: LangGraph checkpointer per thread_id; last 20 messages sent (never starting on an orphan tool result); the user's name is extracted into state and injected each turn, so it survives the window.
- **Loop safety**: 4 tool calls per turn, then the model must answer without tools. Tool errors are returned as text.
- **Answer check**: every number in the answer must come from the document, the user's messages or a calculator result. Otherwise the answer is removed from memory and the model retries once.
- **Live eval** (11 cases, scored on the first sentence): tool precision/recall 1.0/1.0, memory 2/2, accuracy 10/11 → **11/11** with the check. The miss: the model used the tool for 4·12·0.8, then subtracted in its head ("save 6.40" instead of 9.60).

Trade-offs and known limits

- BM25 over embeddings: deterministic, no download, exact terms ("annual", "503"); swap point is `KnowledgeBase.search`. Rules favour recall: false escalations are cheap, missed breaches are not.
- Non-refund claims are only number-checked (e07 stated an unlisted feature limit). The fix is to widen the judge to all sentences, at +1 call per email. The judge is the same small model, so it fails closed.
- Local 9B on one laptop GPU: classify p95 9.5 s, 26 s per email end to end (p95) with 4 workers. Hosted models or a queue + autoscaled workers for real load. MemorySaver → SQLite/Postgres checkpointer for production.

Testing: 85 offline tests with a scripted fake model (every graph path, fail-closed cases, the exact live-run hallucinations, scraper fixture with every kind of clutter, calculator safety, memory window). Raw live outputs in docs/evidence/.